



CEBU INSTITUTE OF TECHNOLOGY
U N I V E R S I T Y

IT342-Section SYSTEMS INTEGRATION AND ARCHITECTURE 1

FUNCTIONAL REQUIREMENTS SPECIFICATION (FRS)

Project Title: Mini App – User Registration & Authentication System

Prepared By: Raymart N. Ruperez

Date of Submission: February 11, 2026

Version: 1.0

Table of Contents

1.	Introduction	3
1.1.	Purpose	3
1.2.	Scope	3
1.3.	Definitions, Acronyms, and Abbreviations	3
2.	Overall Description	3
2.1.	System Perspective	3
2.2.	User Classes and Characteristics	3
2.3.	Operating Environment	3
2.4.	Assumptions and Dependencies	3
3.	System Features and Functional Requirements	3
3.1.	Feature 1:	3
3.2.	Feature 2:	3
4.	Non-Functional Requirements	3
5.	System Models (Diagrams)	4
5.1.	ERD	4
5.2.	Use Case Diagram	4
5.3.	Activity Diagram	4
5.4.	Class Diagram	4
5.5.	Sequence Diagram	4
6.	Appendices	4

1. Introduction

1.1. Purpose

The purpose of this document is to outline the functional and non-functional requirements of the Mini App – User Registration & Authentication System. It is intended for students, instructors, and developers who will design, analyze, and eventually implement the system using ReactJS, Spring Boot, and MySQL.

1.2. Scope

The system offers basic user account management features such as account registration, login, logout, and access to a secured user dashboard. It ensures that only authenticated users can view protected pages. This document concentrates exclusively on system documentation and diagrams and does not cover code implementation.

1.3. Definitions, Acronyms, and Abbreviations

- **FRS** – Functional Requirements Specification
- **UI** – User Interface
- **API** – Application Programming Interface
- **ERD** – Entity Relationship Diagram
- **JWT** – JSON Web Token
- **CRUD** – Create, Read, Update, Delete

2. Overall Description

2.1. System Perspective

The system is a client-server application consisting of a React-based frontend, a Spring Boot backend API, and a MYSQL database. The frontend manages user interactions, the backend handles authentication logic, and MySQL stores user credentials and profile information.

2.2. User Classes and Characteristics

Guest User

- Does not have an account or is not currently logged in
- Can create a new account
- Can log in to the system

Authenticated User

- Has successfully logged in
- Can access a personal dashboard or profile
- Can log out of the system

2.3. Operating Environment

- **Frontend:** ReactJS running on modern web browsers
- **Backend:** Spring Boot REST API
- **Database:** MySQL (for user credentials and profile data)
- **Tools:** draw.io / diagrams.net for diagram creation

2.4. Assumptions and Dependencies

- Users have access to the internet.
- Firebase services are available and correctly configured.
- The backend API is accessible from the frontend.
- Passwords are securely hashed before being stored.

3. System Features and Functional Requirements

3.1. Feature 1:

Description: Allows a guest user to create a new account by submitting valid personal and login details.

Functional Requirements:

- The system shall enable guest users to register using an email address and password.
- The system shall validate all required input fields.
- The system shall save user information to the database after successful registration.

3.2. Feature 2:

Description: Enables registered users to log in, access secured areas of the system, and safely log out.

Functional Requirements:

- The system shall verify user identity using valid login credentials.
- The system shall issue an authentication token after a successful login.
- The system shall prevent unauthenticated users from accessing protected pages.
- The system shall terminate the session or invalidate the authentication token upon logout.

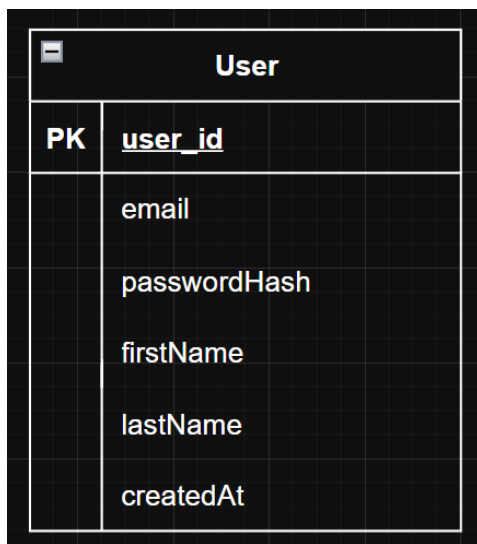
4. Non-Functional Requirements

- **Security:** Passwords must be securely encrypted, and authentication tokens must be properly protected.
- **Performance:** Login and registration processes should respond within acceptable timeframes.
- **Usability:** The user interface should be simple, intuitive, and easy to use.
- **Reliability:** The system should gracefully manage invalid inputs and authentication errors.

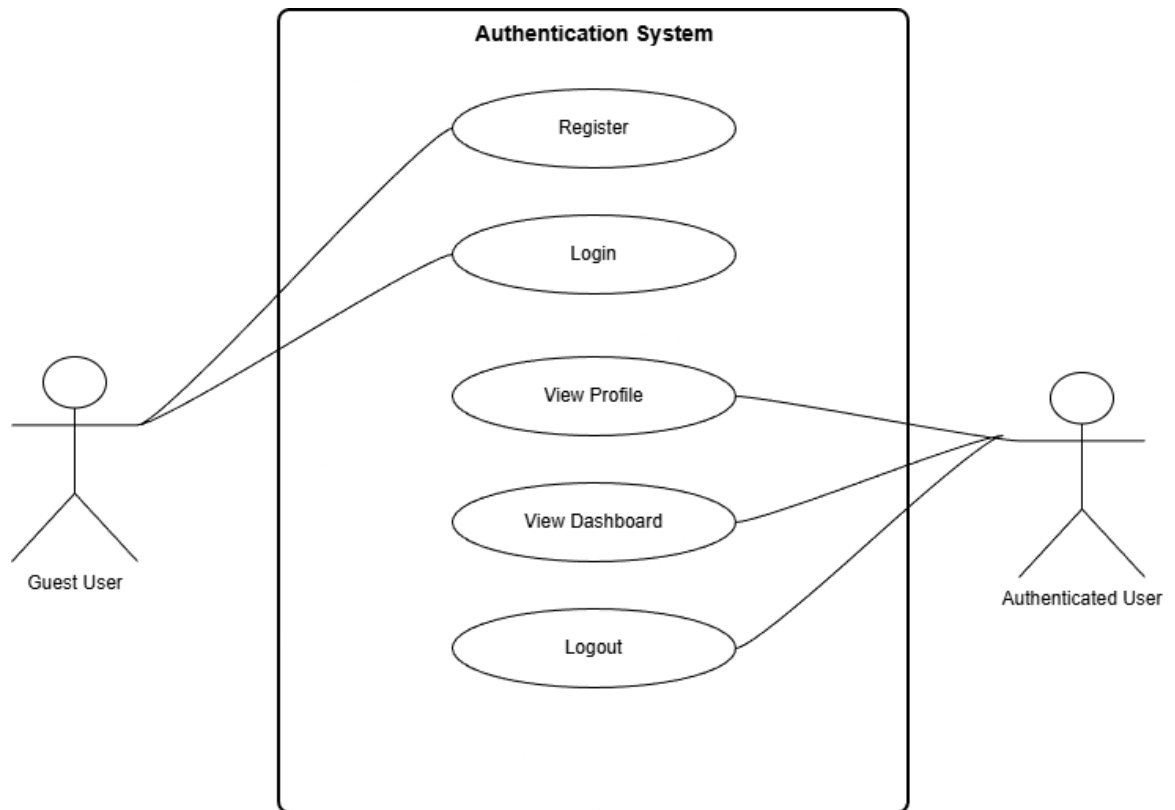
5. System Models (Diagrams)

Insert the necessary diagrams for the system:

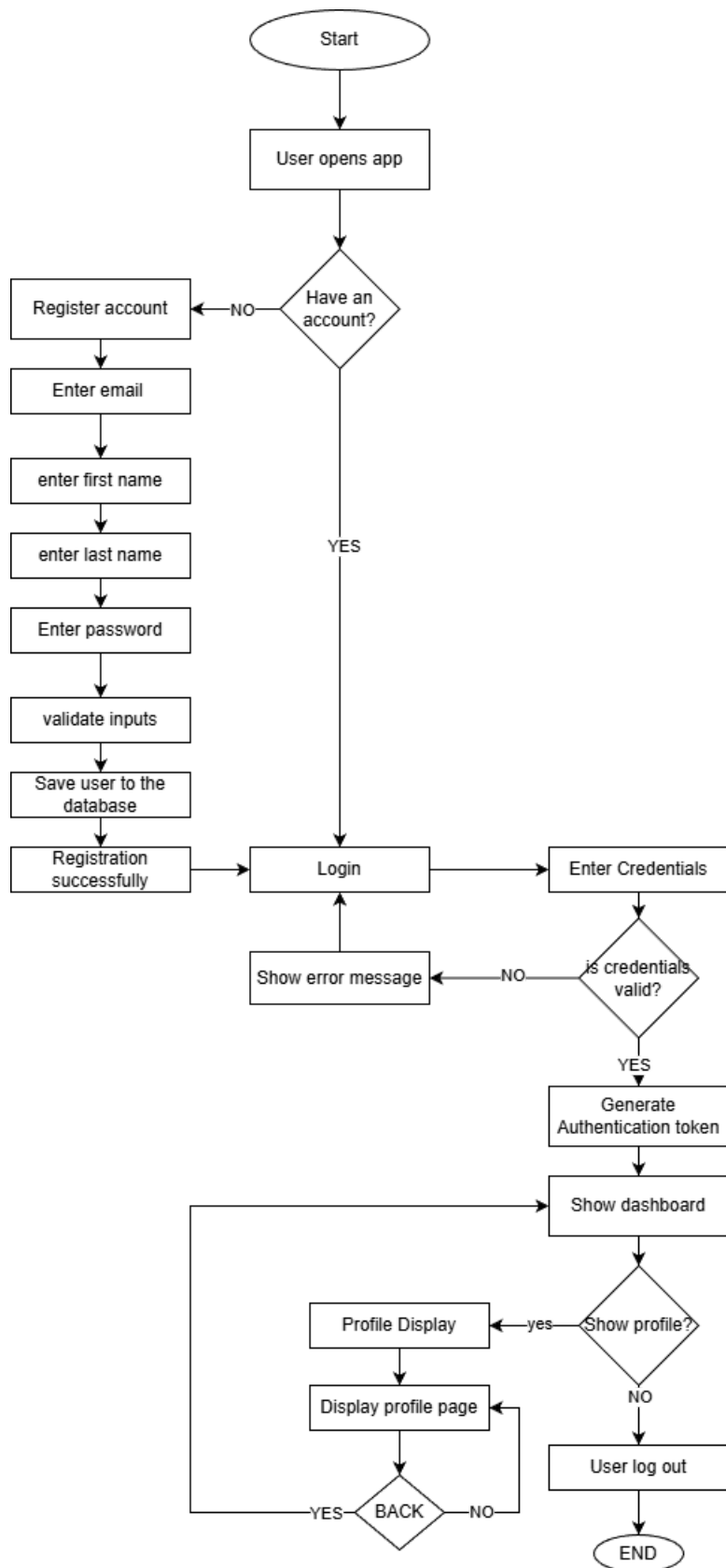
5.1. ERD



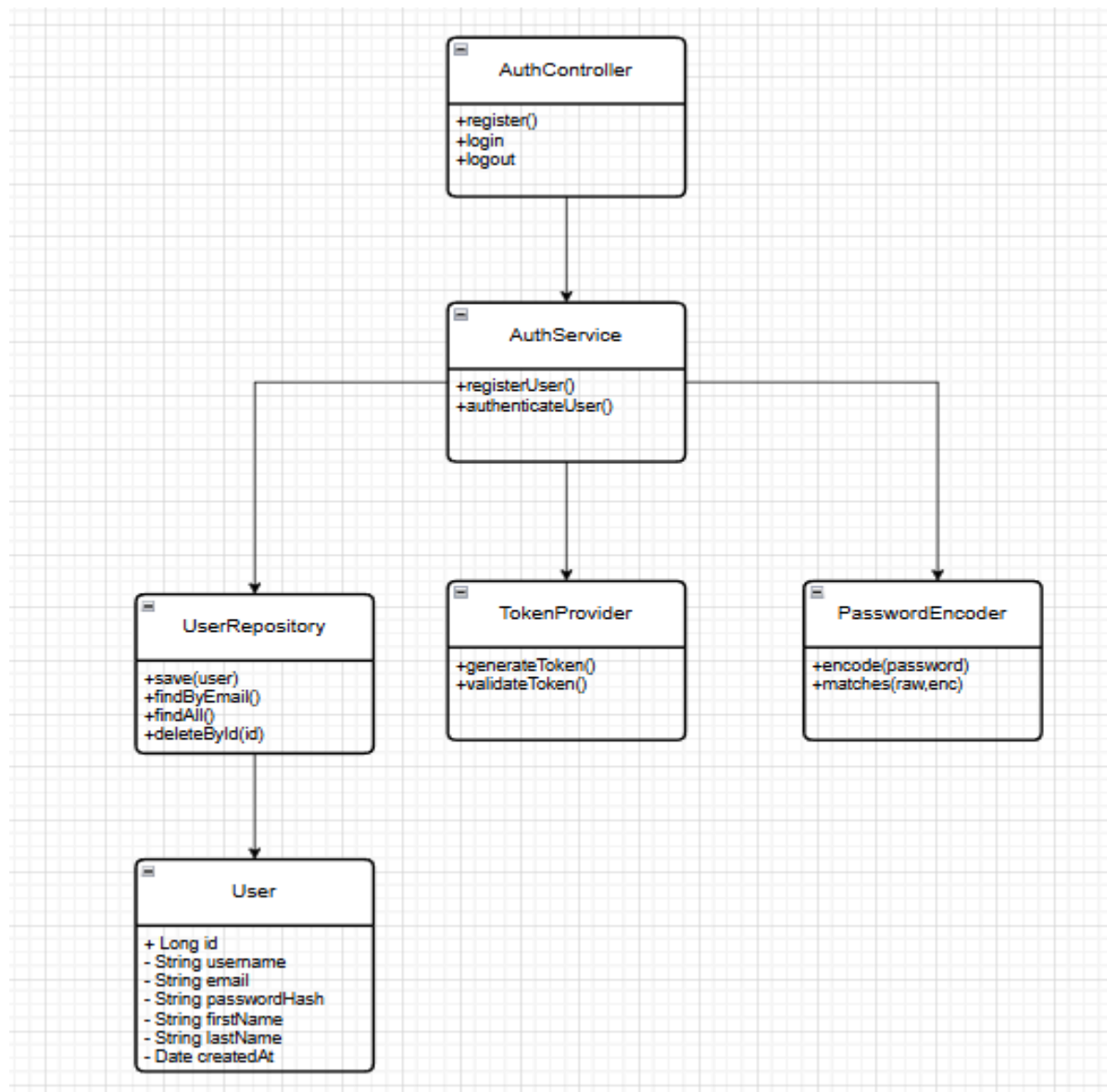
5.2. Use Case Diagram



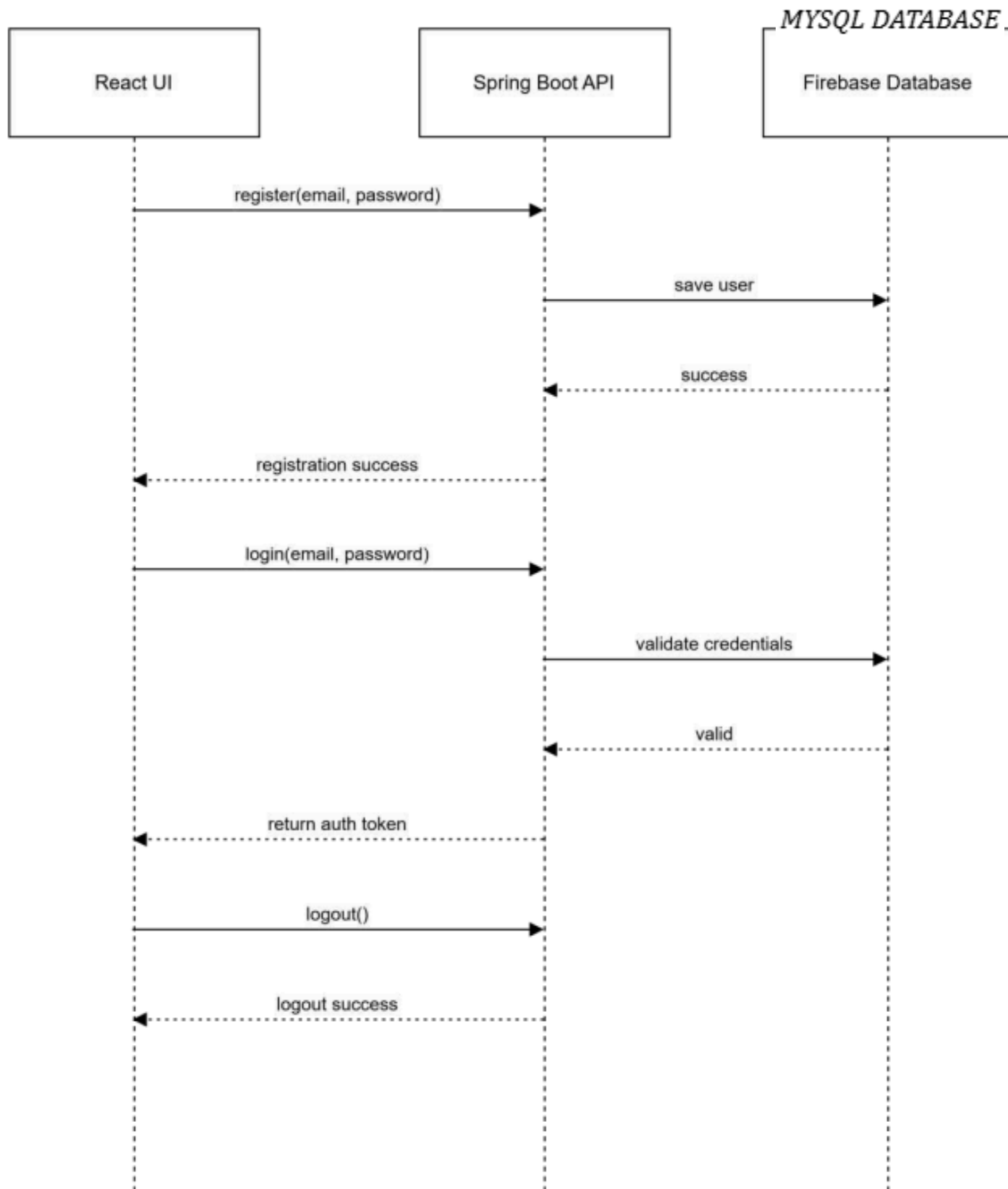
5.3. Activity Diagram



5.4. Class Diagram



5.5. Sequence Diagram



6. Appendices

This document serves as the official reference for the system implementation to be carried out in the succeeding laboratory activity.